

Engineering Principles and Practices I (CG1111A)

Project Report

Studio Group Number: B02

Section Number: 3

Team Number: 2

Name	Student No.
JUSTIN POON JUNJIE	A0308159Y
KARTHIK KATHIRESH	A0308682X
KARTHIKEYAN VETRIVEL	A0307459W
KELVIN KUOK THAI LUK	A0308947N

Date of submission: 17 / 11 / 2024

Academic Year & Semester

For AY24/25 Semester 1

Table of Contents

1 | Introduction: The A-Maze-Ing Race Project **4**

2 | Photographs of mBot and Breadboard Circuits **5**

3 | Overall Circuit Design **6**

3.1 Tinker CAD Rendering of Circuit Design **6**

3.2 HD74LS139P 2-to-4 Decoder IC Chip **7**

3.3 Connections with RJ25 Adapters **8**

4 | Design and Circuitry **9**

4.1 Colour Detection Circuit **9**

4.1.1 Current Limiting Resistors for LEDs 9

4.1.2 Current Limiting Resistors for LDR 11

4.1.3 Final Circuit Design 11

4.2 IR Sensor Circuit **12**

4.2.1 Objective of Circuit 12

4.2.2 Current Limiting Resistor for IR Emitter 12

4.2.3 Current Limiting Resistor for IR Detector 13

4.2.4 Final Design of Circuit 13

5 | Design Considerations **14**

5.1 Colour Sensing Circuit **14**

5.2 Ultrasonic Sensor **15**

5.3 Infrared Sensor Circuit **15**

5.4 Line Follower **16**

5.5 Wiring **16**

6 | Algorithm **17**

6.1 Overview **17**

6.2 Navigation **17**

6.3 Colour Detection **18**

7 | Code Analysis **19**

7.1 Introduction **19**

7.2 Ultrasonic Sensor and Implementation of PID **21**

7.2.1 Understanding Ultrasonic Sensor Fundamentals 21

7.2.2 Obtaining Ultrasonic Distance 21
7.2.3 Initial Approach to Navigate 22
7.2.4 Implementation of PID 22
7.2.5 Ziegler-Nichols Closed Loop Method 24
7.2.6 Final Code for PID 26
7.3 IR Proximity Sensing and Calculations **27**
7.3.1 Understanding IR Sensor Fundamentals 27
7.3.2 Obtaining IR Distance 27
7.3.3 Initial Approach to Navigate 28
7.3.4 Implementation of Ambient IR 29
7.3.5 Final Code for IR Proximity Sensing 30
7.4 Colour Detections and Implementing K-Nearest Neighbours **31**
7.4.1 Obtaining RGB values 31
7.4.2 Initial Approach 32
7.4.3 K-Nearest Neighbours (KNN) Implementation 32
7.4.4 Calibration 34
7.4.5 Final Code 36
7.5 Movement of mBot **37**
7.5.1 Black Strip has been detected 37
7.5.2 Black Strip has not been detected 40
7.6 Celebrate **41**
8 | Difficulties and Resolutions **42**
8.1 The Great Overflow **42**
8.1.1 Problem 42
8.1.2 Solution 42
8.1.3 Reflections 42
8.2 The Faulty Foot **43**
8.2.1 Problem 43
8.2.2 Solution 43
8.2.3 Reflections 43
8.3 The Battery Betrayal **44**
8.3.1 Problem 44
8.3.2 Solution 44
8.3.3 Reflections 44
9 | Work division of the project **45**
10 | Conclusion **46**

1 | Introduction: The A-Maze-Ing Race Project

In this project, we were tasked with designing an autonomous robot (mBot) that can navigate a maze successfully. An example of a maze layout is shown below in Figure 1.1.

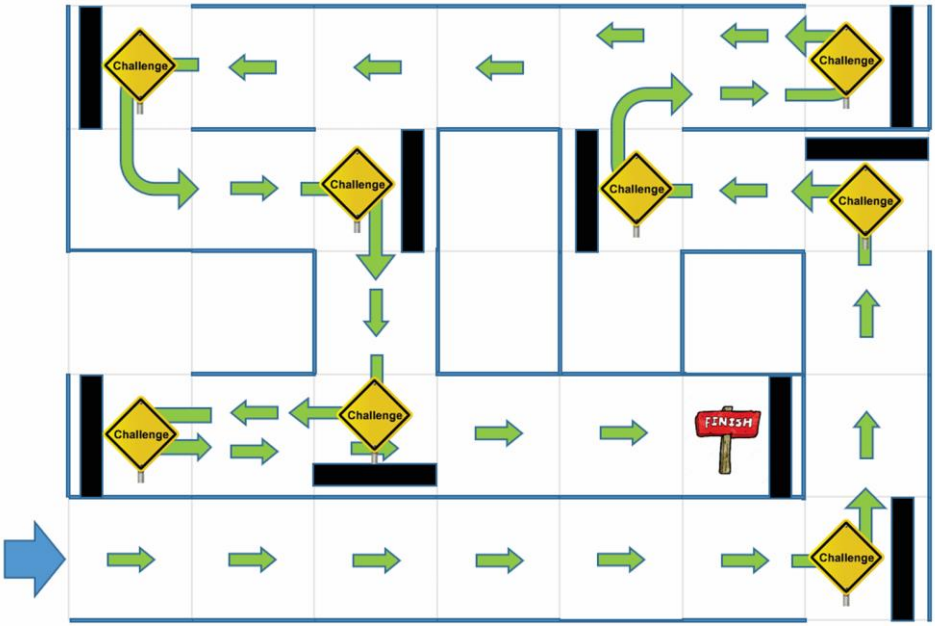


Figure 1.1: Sample Maze

While navigating the maze, the mBot was required to resolve multiple waypoint challenges. At each challenge, the mBot stops upon detecting a black strip of paper. Then, it had to execute a directional change based on the colour of the paper that is placed beneath it. This is achieved by using a Colour Sensing Circuit. Different colours denoted different turns to be made by the mBot. Once the mBot reaches the end of the maze (as denoted by a white paper), it was required to stop and play a celebratory tune.

There were several project requirements regarding the mBot's general movement. The mBot was required to travel in a straight line as much as possible, without bumping into any walls. When making turns at the waypoint challenges, the mBot was required to be as precise as possible, by not over- or under-manoeuvring too much. These were achieved by using an Ultrasonic (US) Sensor and an Infrared (IR) Sensor, which were placed on each side of the mBot.

There were two major components in this project - designing the hardware components of the mBot (i.e. circuitry) and developing the software algorithms for running the mBot. Using the knowledge gained over the past 12 Studios, we worked on these aspects, to achieve the desired outcomes of this project.

2 | Photographs of mBot and Breadboard Circuits

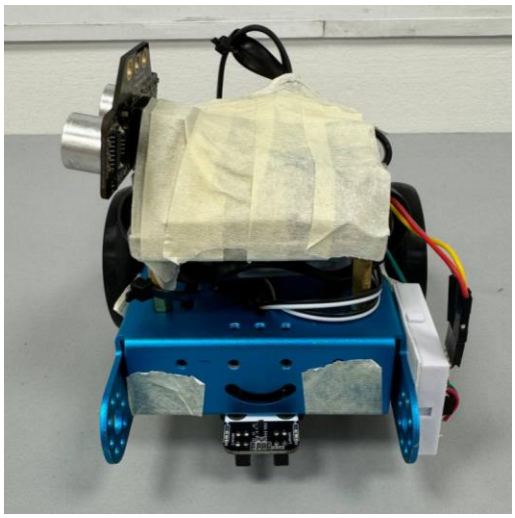


Figure 2.1: Front View

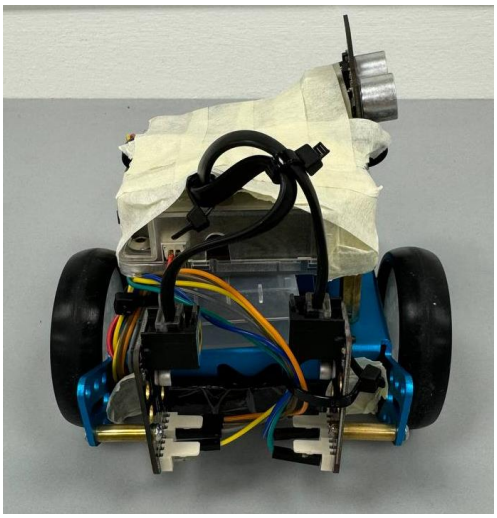


Figure 2.2: Back View

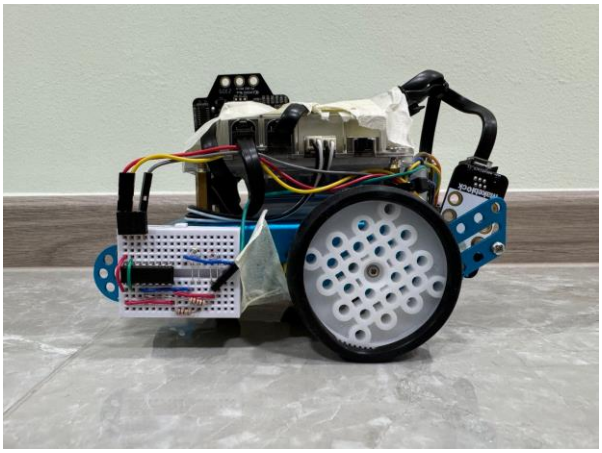


Figure 2.3: Side-View (Left)

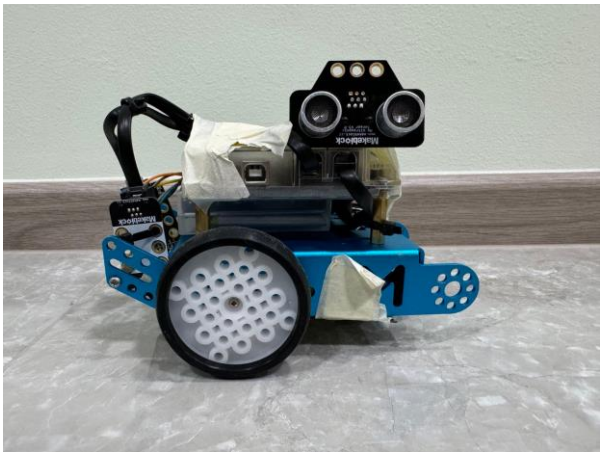


Figure 2.4: Side-View (Right)

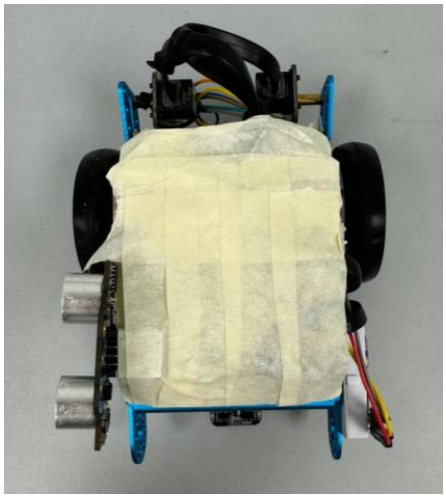


Figure 2.5: Top View

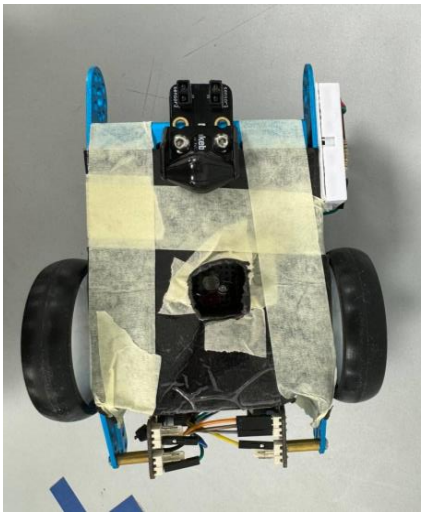


Figure 2.6: Bottom View

3 | Overall Circuit Design

This section will outline the layout of our circuit design and our choices, before explaining them in detail in the following subsections.

3.1 Tinker CAD Rendering of Circuit Design

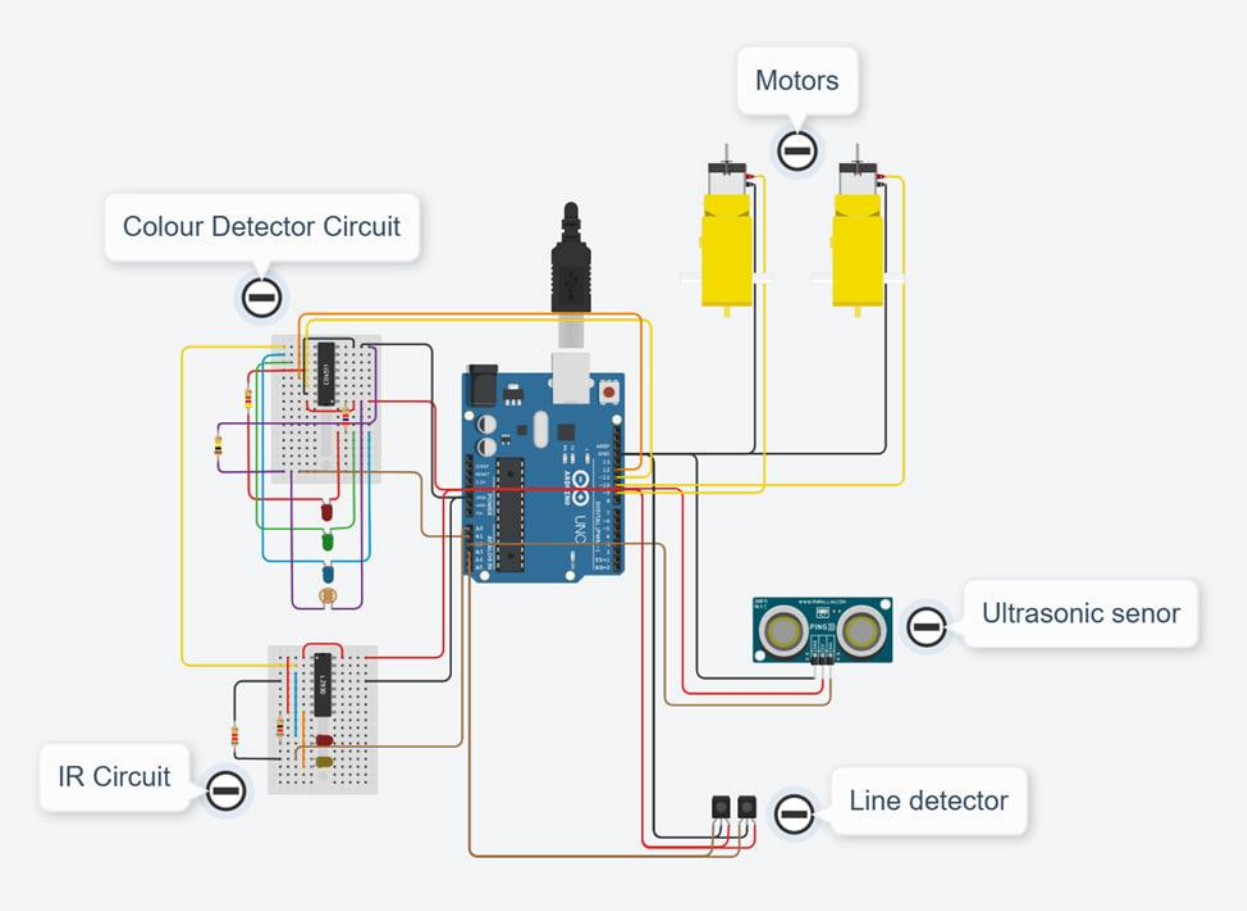


Figure 3.1: Tinker CAD Design

3.2 HD74LS139P 2-to-4 Decoder IC Chip

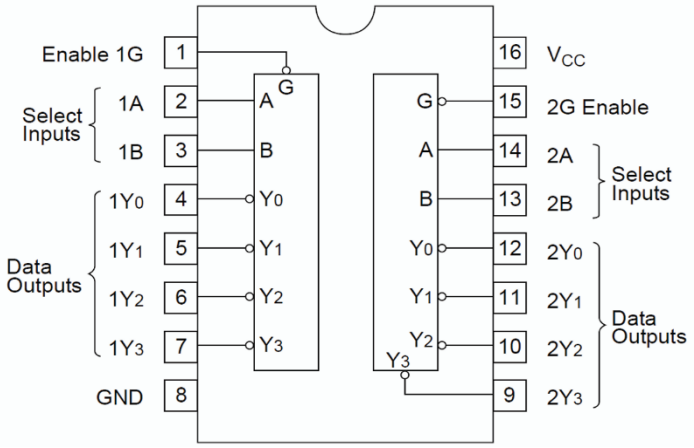


Figure 3.2: Pin Layout of HD74LS139P 2-to-4 decoder IC chip

This project required only one set of 2-to-4 decoders. Referring to Figure 3.2, we chose the second decoder, which uses Pins 9 – 15. Table 3.1 illustrates our choices regarding the connections, and Table 3.2 summarizes the behaviour of the IC Chip.

Pin	Connection
2A	J4 (S1)
2B	J4 (S2)
Y_0	Red LED
Y_1	Green LED
Y_2	Blue LED
Y_3	IR Emitter

Table 3.1: Pin Choices

Inputs			Outputs				Component
G	2A	2B	Y_0	Y_1	Y_2	Y_3	
L	L	L	L	H	H	H	Red LED
L	L	H	H	L	H	H	Green LED
L	H	L	H	H	L	H	Blue LED
L	H	H	H	H	H	L	IR Emitter

Table 3.2: Behaviour of the HD74LS139P 2-to-4 Decoder

3.3 Connections with RJ25 Adapters

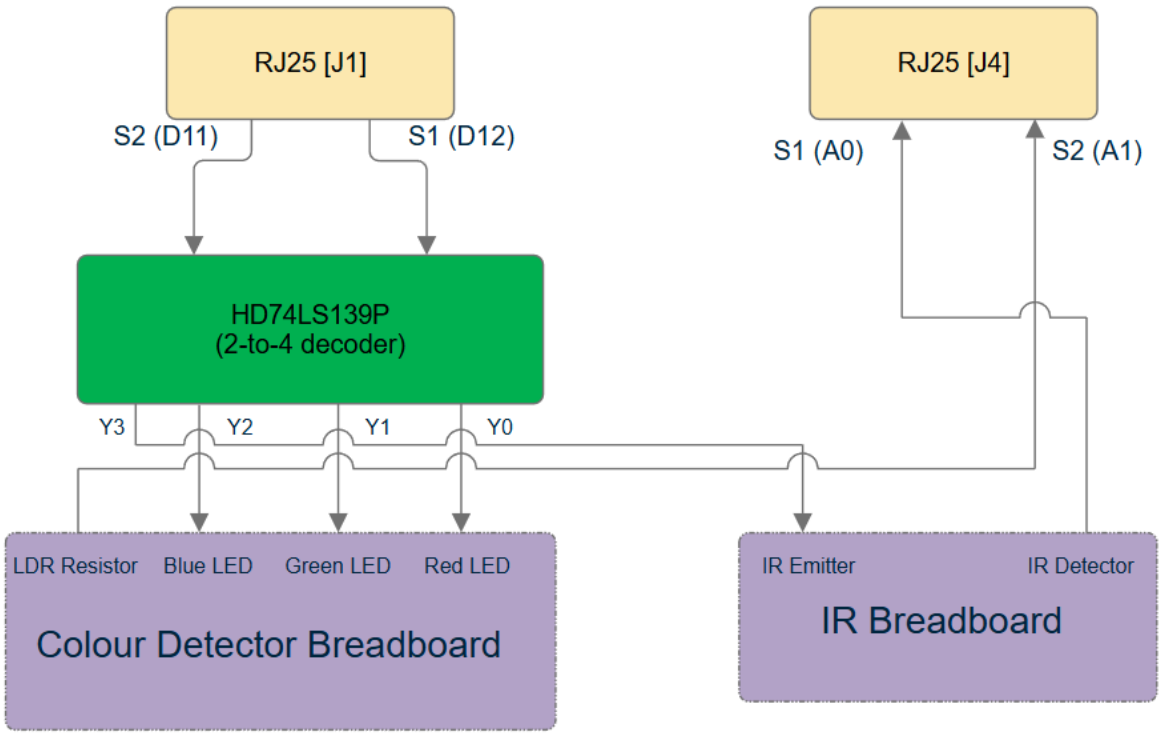


Figure 3.3: Connection with RJ25 Adapters

Figure 3.3 illustrates the circuit connection between the Arduino Pins and the 2-to-4 Decoder IC chip via the RJ25 adapters.

4 | Design and Circuitry

In this section, we will explain the different circuits built on the mBot.

4.1 Colour Detection Circuit

This circuit is used to identify the colour of the paper placed at the waypoint challenges at different parts of the maze. Upon sensing the colour, the mBot will execute the required turns.

4.1.1 Current Limiting Resistors for LEDs

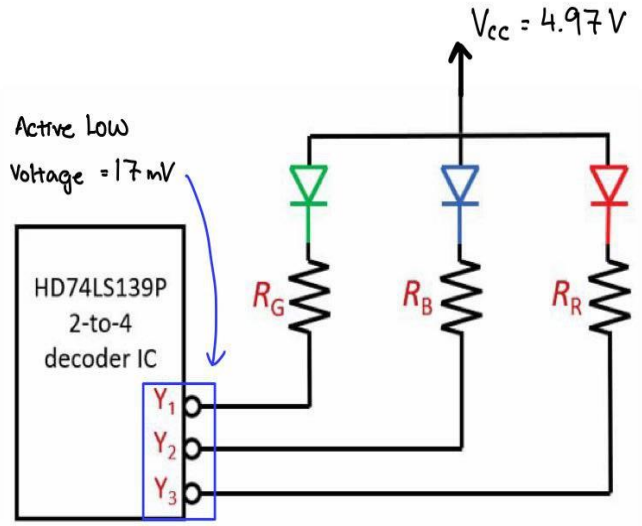


Figure 4.1: LED circuit connected to HD74LS139P Chip

Firstly, we measure the values for V_{cc} and the active low voltage of the HD74LS139P Chip to accurately calculate the required resistance needed for each LED. Since the current limit for each of the data output pins is 8 mA, we decided to design a circuit with a current of 7 mA flowing through each LED.

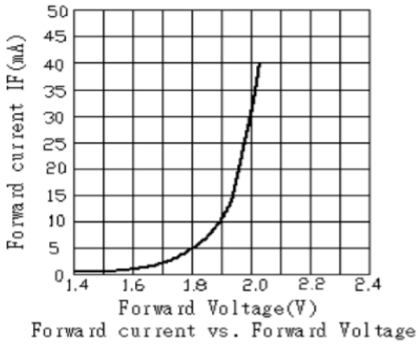


Figure 4.2: I – V Characteristic Graph for RED LED

$$Referring\ to\ Figure\ 4.2,$$

$$When\ I_R = 7\ mA,$$

$$V_{FORWARD\ (RED)} \approx 1.9\ V$$

$$V_{RESISTOR} = (4.97\ V - 0.17\ V) - 1.9\ V = 2.9\ V$$

$$Since\ V_{RESISTOR} = I \times R_{RESISTOR},$$

$$I \times R_{RESISTOR} = 2.9\ V, where\ I = 7 \times 10^{-3}\ A$$

$$R_{RESISTOR} = \frac{2.9}{7 \times 10^{-3}} \approx 414.29\ \Omega$$

$$To\ get\ a\ maximum\ current\ of\ 7\ mA, R_{RESISTOR}\ needs\ to\ be\ more\ than\ 414.29\ \Omega.$$

$$\boldsymbol{Therefore, R_{RESISTOR} > 414.29\ \Omega}$$

Based on these calculations, we tried to use a 420 Ω resistor for the Red LED since it was the closest to the calculated value above.

By repeating the previous steps, we identified the current limiting resistors for both the Blue LED and green LED.

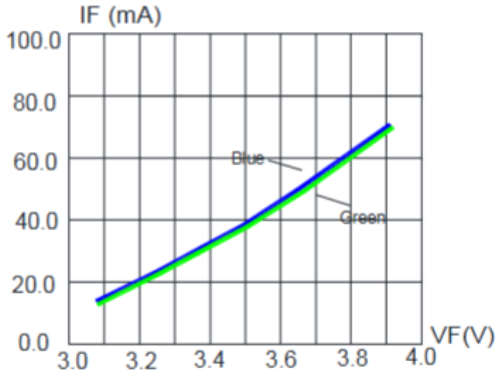


Figure 4.3: I – V characteristics of Blue LED and Green LED

For a current of 7 mA, both the Blue LED and Green LED would have a forward voltage of approximately 3.0 V.	
Blue	***Green***
$V_{FORWARD\ (BLUE)} = 3.0\ V$ $V_{RESISTOR} = (4.97\ V - 0.17\ V) - 3\ V = 1.8\ V$ $V_{RESISTOR} = I \times R_{RESISTOR}$ $R_{RESISTOR} = \frac{1.8}{7 \times 10^{-3}} \approx 257.14\ \Omega$	$V_{FORWARD\ (GREEN)} = 3.0\ V$ $V_{RESISTOR} = (4.97\ V - 0.17\ V) - 3\ V = 1.8\ V$ $V_{RESISTOR} = I \times R_{RESISTOR}$ $R_{RESISTOR} = \frac{1.8}{7 \times 10^{-3}} \approx 257.14\ \Omega$
$\boldsymbol{R_{RESISTOR} > 257.14\ \Omega}$	$\boldsymbol{R_{RESISTOR} > 257.14\ \Omega}$

Based on these calculations, to get a maximum current of 7 mA for both the blue and green LEDs, the resistors should be of a value higher than 257.14 Ω. Thus, we used a 330 Ω resistor which is the closest resistor value available in the lab.

However, the calculation of the resistors is the minimum resistance required to ensure that the maximum current in the 2-to-4 decoder chip is less than 8 mA.

As we started testing our colour detection circuit, we realised that all three LEDs are shining too bright. Through trial and error, we identified that a 3060 Ω resistor is best suited for the Red LED, while a 2.6kΩ resistor is best suited for both the Blue and Green LEDs.

4.1.2 Current Limiting Resistors for LDR

For our LDR, we chose a 100 kΩ resistor, as it not only had been previously used in our studios, but also provided reliable results.

4.1.3 Final Circuit Design

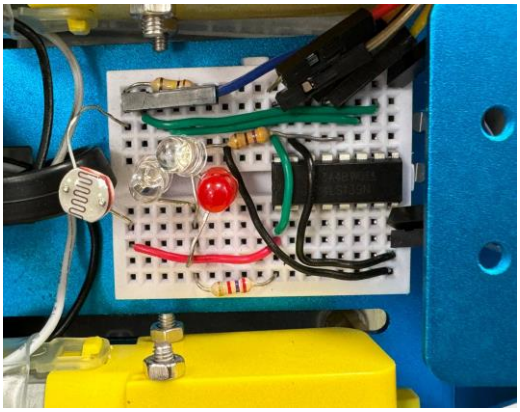


Figure 4.4: Final Colour Sensing Circuit Design

4.2 IR Sensor Circuit

4.2.1 Objective of Circuit

The goal of the Infrared (IR) circuit is to make use of the indirect incidence setup to measure the distance between the mBot and the wall, preventing the mBot from colliding with the maze wall on one side. For our mBot, the circuit is mounted on the left to detect incoming walls.

4.2.2 Current Limiting Resistor for IR Emitter

ABSOLUTE MAXIMUM RATINGS AT TA=25℃

PARAMETER	MAXIMUM RATING	UNIT
Power Dissipation	75	mW
Peak Forward Current (300pps, 10 μ s pulse)	1	A
Continuous Forward Current	50	mA
Reverse Voltage	5	V
Operating Temperature Range	-40℃ to + 85℃	
Storage Temperature Range	-55℃ to + 100℃	
Lead Soldering Temperature [1.6mm(.063") From Body]	260℃ for 5 Seconds	

Figure 4.5: IR Sensor Datasheet

According to Figure 4.6, the maximum rated continuous forward current is 50 mA. Thus, we can calculate the minimum resistor value we can use for our current limiting resistor.

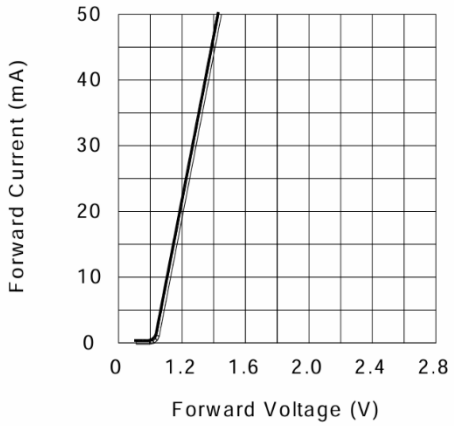


Figure 4.6: I – V Characteristic Graph of IR Emitter

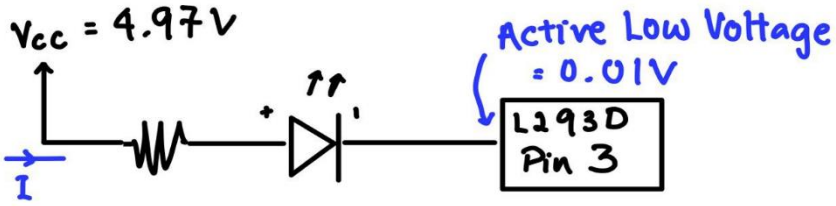


Figure 4.7: IR circuit connected to L293D Chip

$$Referring\ to\ Figure\ 4.7,$$

$$V_{FORWARD\ (IR\ EMITTER)} \approx 1.4\,V$$

$$To\ find\ the\ Voltage\ across\ the\ Resistor,$$

$$V_{RESISTOR} = (4.97\,V - 0.01\,V) - 1.4\,V = 3.56\,V$$

$$Since\ V_{RESISTOR} = I \times R_{RESISTOR},$$

$$R_{RESISTOR} = \frac{3.56}{50 \times 10^{-3}} \approx 71.2\,\Omega$$

$$\boldsymbol{Therefore, the\ minimum\ resistance\ needed\ is\ 71.2\ \Omega}\ .$$

We decided to use an 82 Ω resistor as it is the closest to our calculated value that satisfies the maximum rated continuous forward current. We wanted to maximize the current flowing through the IR emitter so that we maximize the intensity of the emitted IR. We believe that a stronger signal will yield more accurate readings since it improves the signal-to-noise ratio, making it easier for the sensor to detect variations accurately.

4.2.3 Current Limiting Resistor for IR Detector

For our IR Detector resistor, we used an 8.2 kΩ resistor not only because it was previously used in studio 12, but also because it gives us reliable readings.

4.2.4 Final Design of Circuit

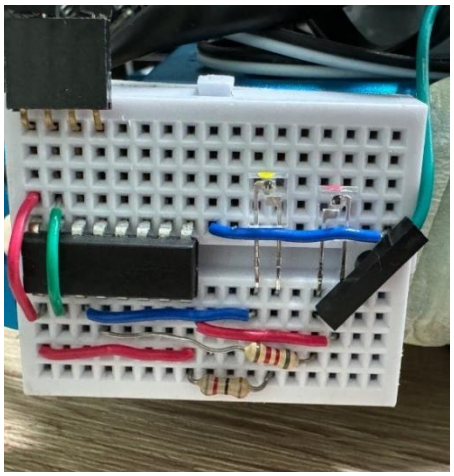


Figure 4.8: IR Proximity Sensor Circuit

5 | Design Considerations

In previous sections, we explained the different circuits and choices made. In this section, we will discuss the design choices made pertaining to the placement of the circuits and components on the mBot.

5.1 Colour Sensing Circuit

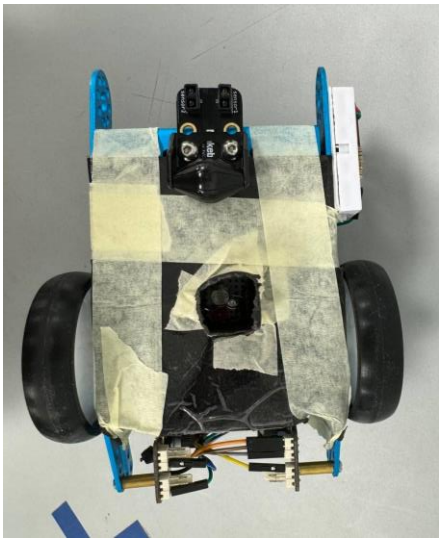


Figure 5.1: Top-View of Shielding

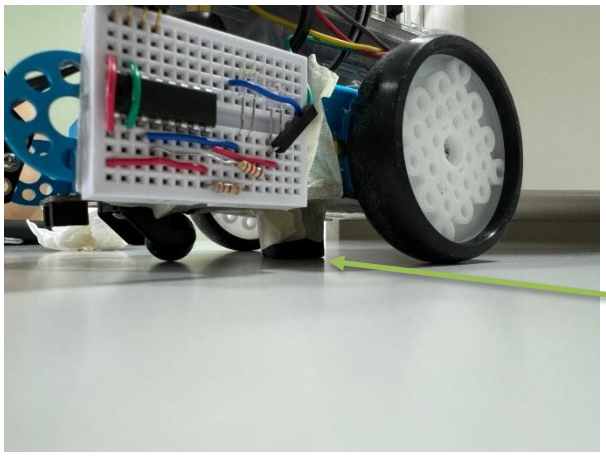


Figure 5.2: Side-View of Shielding

By default, the colour sensing circuit was placed at the bottom of the mBot, so that it can achieve its purpose of sensing the colour of the paper beneath the mBot. However, just fixing the breadboard at the bottom of the mBot, was not sufficient. Due to the interference of natural light and other light sources, the readings captured by the LDR sensor can become largely inaccurate. To mitigate this, we added a shielding to the mBot. As shown in Figure 5.1, we taped up the bottom of the mBot entirely using black paper, to reduce the interference of external light sources. We also taped up any other 'holes' in the mBot that may allow the LDR sensor to read any external light sources. Next, as shown in Figure 5.2, we cut a hole in the black paper, and inserted a 'tunnel' through it, that allows only the Red LED, Green LED, Blue LED and LDR sensor to be exposed to external light conditions. These elements largely minimized any inaccuracies inflicted by external light conditions, ensuring that the LDR sensor readings were mostly accurate.

5.2 Ultrasonic Sensor

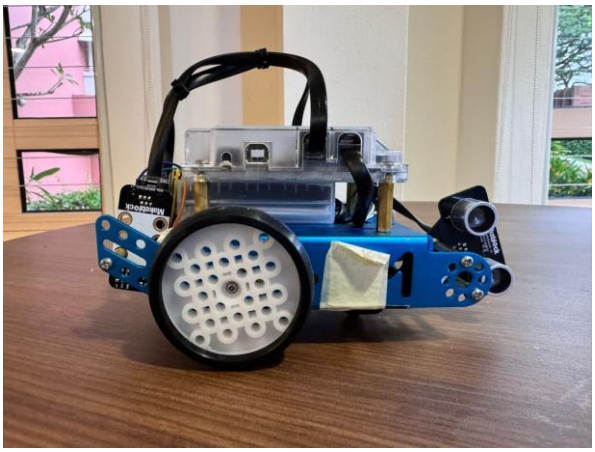


Figure 5.3: Initial Placement of Ultrasonic Sensor

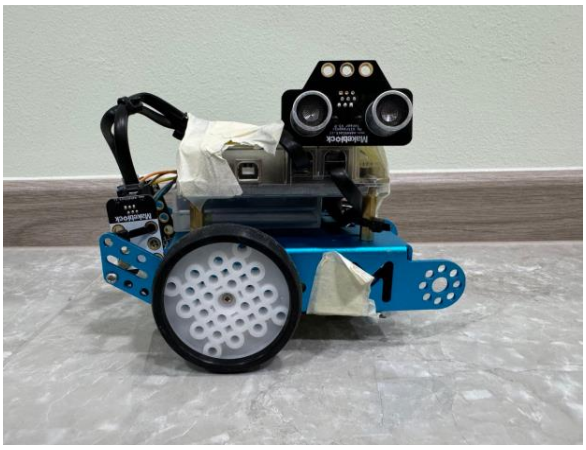


Figure 5.4: Final Placement of Ultrasonic Sensor

We positioned the Ultrasonic Sensor on the right-side of the mBot, so that it can detect the proximity of walls on its right, as it navigates the maze. Figure 5.3 shows the initial position of the sensor. After a few trial runs, we identified that this position leads to inaccurate readings. When there is no wall on the right-side, instead of detecting an “infinite” distance (quantified as 400 cm), the sensor detects the curb along the edges of the maze table. Due to the divergent nature of ultrasonic waves, there was a possibility that the waves rebounded on the curb of the table and gave us inaccurate readings. To mitigate this, we changed the position of the sensor, as shown in Figure 5.4. This allowed for more accurate readings, as the sensor reads distances solely between the mBot and the white walls.

5.3 Infrared Sensor Circuit

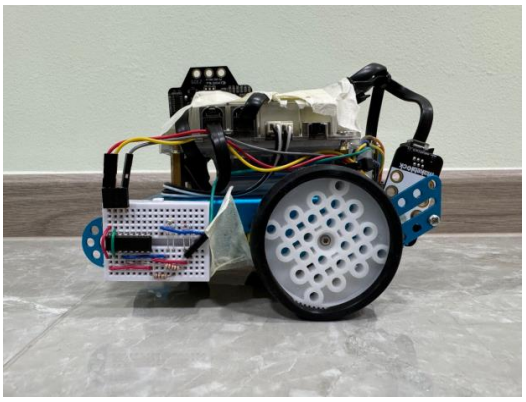


Figure 5.5: Placement of IR Sensor Circuit

The IR sensor circuit was positioned on the left-side of the mBot, to detect the proximity of walls on the left. We maintained this position throughout the project duration, as it provided accurate readings.

5.4 Line Follower



Figure 5.6: Placement of Line Follower Sensor

The line follower sensor was positioned at the front of the mBot, so that it can detect the black strip at waypoint challenges and come to a stop. As shown in Figure 5.6, we positioned the sensor such that that the mBot stops with sufficient clearance between itself and the front-facing wall. This is to ensure it does not collide the wall when stopping at waypoint challenges.

5.5 Wiring

We spent some meticulous effort in our wire management, to ensure that they do not interfere with the movement of the mBot, and the readings from both the Ultrasonic and IR Sensors. Using zip-ties, we bunched up groups of wires together. As for the RJ25 adapters, we positioned them as shown in Figure 5.8 below. We made sure to not attach the adapters directly to the metal body, to avoid shorting the pins when the soldering joints make contact with the metal body.

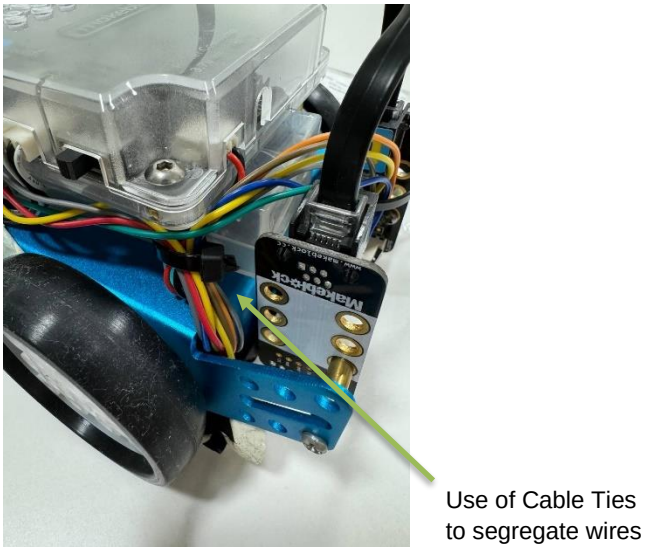


Figure 5.7: Wire Management

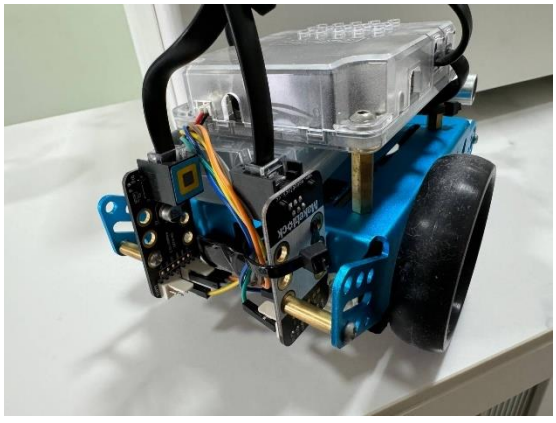


Figure 5.8: Placement of RJ25 Adapters

6 | Algorithm

6.1 Overview

The operation of our mBot can be divided into 2 main components: **Navigation and Colour Detection**. This section provides a concise overview on the sequence of how our mBot successfully navigates and clears the maze.

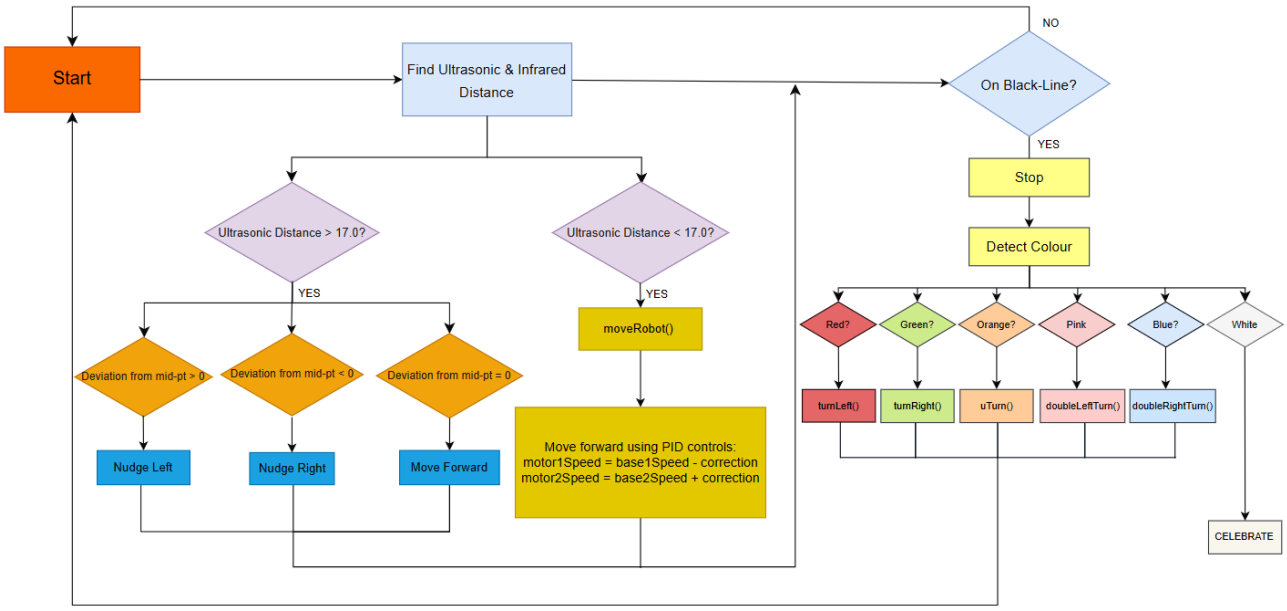


Figure 6.1: Algorithm Flowchart

6.2 Navigation

To navigate the maze smoothly, we had to ensure that the mBot does not collide with any walls, and is able to deflect off from the wall, when too close.

To achieve a smooth and stable navigation, we decided to implement the Proportional-Integral-Derivative (PID) control in our navigation systems. Upon testing and comparing both the Ultrasonic sensor and Infrared Detector, we concluded that the Ultrasonic Sensor is significantly more accurate *(refer to Sections 7.2 and 7.3)*. Therefore, we used it as our primary sensor for PID control.

When there are no walls for the Ultrasonic Sensor to detect, the proximity sensing algorithm switches to the use of the Infrared Detector. Using the Analog pin (A1), we scaled the sensor readings to distances in centimetres *(refer to section 7.3.2: Obtaining IR Distance)*. We then established a deviation boundary from the mid-point to ensure that the mBot is centered within the path. When the deviation is positive, the mBot is too far to the right and needs to nudge back left, and likewise, when the deviation is negative, the mBot is too far to the left and it needs to

nudge back right. Nudging refers to the mBot re-orienting itself back to a straight path along the maze.

6.3 Colour Detection

Upon detection of the black strip of paper at waypoint challenges, the mBot immediately stops, initiating the colour detection process. By leveraging an LDR and a 2-to-4 decoder chip to toggle the Red, Green, and Blue LEDs, we measured the voltage changes in the LDR through the Analog Pin (A1). Using the Black Array and Grey Difference Array obtained during the calibration process *(Refer to Section 7.4.1: Obtaining RGB Value)*, we were able to convert these readings to RGB values.

The K-Nearest Neighbours (KNN) algorithm *(refer to Section 7.4.3: K-Nearest Neighbour implementation)* was then employed to accurately determine the colour detected beneath the mBot. The mBot then executes the corresponding actions linked to each colour.

7 | Code Analysis

7.1 Introduction

As a group, we collectively agreed that to write good code, we must ensure that it is **Readable**, **Maintainable** and **Efficient**.

Readable:

To enhance the readability of our code for anyone reviewing it, we made sure that our functions and variables have clear, descriptive names that accurately reflect their purpose. Furthermore, we made sure to include comprehensive comments, so that each function and command can be easily understood at a glance. This approach significantly improved the clarity of the code.

Maintainable:

Having a single file with the entire code made it challenging for us to navigate through the code, and this caused confusion and difficulties when debugging. Therefore, we decided to use separate header files for each segment of our code, to make it easier for us to navigate and maintain the code.

```
#include <MeMCore.h>        // Library for handling mBot hardware components

/************* Self-Made Header Files ***************/
#include "Celebrate.h"      // Library for celebratory functions (e.g., playing a tune)
#include "MotorControls.h"  // Library for motor controls
#include "DetectColour.h"   // Library for detecting colours
#include "SensorValues.h"   // Library for to obtain ultrasonic sensor and infrared sensor values
#include "PID.h"            // Library to control PID movements
```

Figure 7.1.1: Self-Made Header Files

Efficient:

Efficiency was a key consideration in our code, as we needed to ensure that the void loop() function executes as quickly as possible. This was crucial because the mBot must detect the Black line, indicating it to stop, in time. If the code is too resource-intensive, the mBot might miss the line, leading to unintended behaviour. Therefore, we minimised unnecessary delays, removed all Serial.print statements except during debugging, and focused on optimising the overall execution flow.

Moreover, we utilised the method of **parallel arrays** to streamline our implementation and reduce the use of repetitive function calls or complex if-else statements. This is when we use multiple arrays with the same size, where the i-th element in one array is directly related to the i-th element in another array. As shown in Figure 7.1.2, we created a distances[] array to store computed distances, as well as a corresponding colors[] array to represent associated colours. This approach eliminated the need to use multiple if-else statements and made the code more concise and efficient.

```
// Calculate distances between current color and each predefined color
float distances[] = {
  distance(redArray[0], redArray[1], redArray[2], currentColor[0], currentColor[1], currentColor[2]),
  distance(greenArray[0], greenArray[1], greenArray[2], currentColor[0], currentColor[1], currentColor[2]),
  distance(orangeArray[0], orangeArray[1], orangeArray[2], currentColor[0], currentColor[1], currentColor[2]),
  distance(pinkArray[0], pinkArray[1], pinkArray[2], currentColor[0], currentColor[1], currentColor[2]),
  distance(blueArray[0], blueArray[1], blueArray[2], currentColor[0], currentColor[1], currentColor[2]),
  distance(whiteArray[0], whiteArray[1], whiteArray[2], currentColor[0], currentColor[1], currentColor[2])
};

int colors[] = {RED, GREEN, ORANGE, PINK, BLUE, WHITE};  // Array representing color codes

// Find the color with the minimum distance
int minColor = colors[0];      // Initialize to minimum distance and its corresponding minimum colour
float minDist = distances[0];
for (int i = 1; i < 6; i++) {
  if (distances[i] < minDist) {  // Check if current distance is smaller and update accordingly
    minDist = distances[i];
    minColor = colors[i];
  }
}
return minColor;
```

Figure 7.1.2: Implementation of Parallel Arrays

The following sub-sections will further explain the methods employed to uphold good coding standards, as we utilised various algorithms like PID control, IR proximity sensor and K-Nearest Neighbours (KNN) algorithm.

7.2 Ultrasonic Sensor and Implementation of PID

7.2.1 Understanding Ultrasonic Sensor Fundamentals

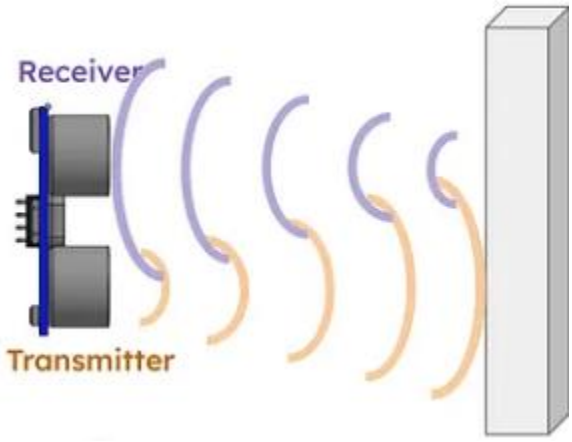


Figure 7.2.1: Ultrasonic Sensor

The ultrasonic sensor is an instrument that measures the distance to an object using ultrasonic waves. This is done by emitting ultrasonic waves using the transmitter and receiving it back using the receiver, and measuring the time taken to do so. This time can then be calculated using the following method:

$$D = v \times \frac{t}{2}$$

where $D = Distance\ between\ ultrasonic\ and\ wall$,
$v = Speed\ of\ sound\ (\sim 340 m/s)$,
$t = Time\ taken\ to\ receive\ back\ ultrasonic\ waves$

Calibration of Ultrasonic Sensor

Upon testing the ultrasonic sensor against different distances, and comparing it with the recorded value, we found that it consistently provided accurate and correct distance readings. Hence, calibration of the ultrasonic sensor was not required.

Real Dist.	Ultrasonic Dist
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9
10	10
11	11

Table 7.1: Ultrasonic Sensor Readings

7.2.2 Obtaining Ultrasonic Distance

The procedure to obtain the distance measured by Ultrasonic Sensor, was simplified using the MeMCore.h library. Firstly, we instantiated an object “ultraSensor” from the “MeUltrasonicSensor”

class, with PORT_3 specified as the connection port. We then implemented the readUltrasonic() function to obtain the distance measured (in centimetres) using the distanceCm() method of the ultraSensor object.

```
/************* Sensor Initialisation ****************/
MeUltrasonicSensor ultraSensor(PORT_3); // Ultrasonic sensor connected to Port 3

/************* Functions to Obtain Ultrasonic and Infrared Values ****************/
float readUltrasonic() {
  int distance_measured = ultraSensor.distanceCm();  // Get distance measurement in cm
  delay(10);                                         // Delay for stability in readings
  return distance_measured;
}
```

Figure 7.2.2: Obtain Ultrasonic Distance

7.2.3 Initial Approach to Navigate

Initially, we attempted to correct the mBot's course, by setting a simple if-else conditional statement, to make it turn slightly when off-track. However, we soon realised that this method was inaccurate. When the mBot was very close to the wall, it would understeer and lead to a collision. Alternatively, we increased the nudge speed, but this resulted in the mBot to oversteer, and it would move in a zig-zag movement about the centre point.

Furthermore, this code lacked elegance and precision, and we decided to implement a more stable and refined solution using a PID controller.

7.2.4 Implementation of PID

The Proportional-Integral-Derivative (PID) feedback controller receives the input data from sensors and calculates the difference between desired position and current position. Upon determining the difference, it induces a correction onto the mBot to try and rectify the error.

In our case, the Ultrasonic Sensor was our primary sensor. Before using PID, we ensured that the ultrasonic sensing was accurate, and it did not produce any unpredicted readings. *(Refer to Section 7.2.1: Understanding Ultrasonic Sensor Fundamentals)* This ensured a controlled movement of the mBot and it was able to centre itself in a smooth and stable manner.

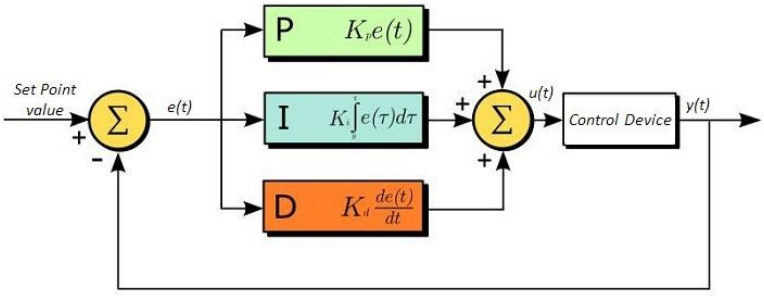


Figure 7.2.3: PID Flowchart

Proportional Component (P)

The Proportional Component (P) determines how fast we want the change of the error correction to happen and is given by the product of the error and proportional gain (K_P).

$$P = K_P \times error(t)$$

A higher K_P would mean that the error correction is stronger, and it takes lesser time for the mBot to return to the center. On the other hand, too strong of a correction will result in an over-correction and cause the mBot to oscillate in an uncontrolled manner.

In our code, the error was obtained by comparing the current heading against the target heading, where the target heading is the mid-point. `[error = targetHeading - currentHeading;]`

Derivative Component (D)

The Derivative Component (D) predicts future error by analysing the rate at which the error is changing. Essentially, it calculates how quickly the error is changing and adjusts the control output accordingly. If the error is changing too quickly, the derivative component applies a correction to slow down this rapid change and dampen the oscillation. The derivative component is the product of the Derivative Gain and the rate of change of error.

$$D = K_D \times \frac{d(error(t))}{dt}$$

A high derivative gain can be effective at reducing oscillations, but it may take a longer time to stabilise at the centre.

In our code, the rate of change of error [$\frac{d(error(t))}{dt}$] was obtained by comparing the current error obtained with previous error, stored as a global variable.

`[derivative = error - previousError;]`

Integral Component (I)

The Integral Component (I) sums the error accumulated over time and helps to correct any bias or offset that might have caused the mBot to drift off-course in the long term. Over time, this integral term will gradually build up a correction to eliminate steady-state error. Therefore, the Integral Component is the product of Integral Gain and the summation of error over time.

$$I = K_I \times \int_0^t e(t)dt$$

On the other hand, having a high integral gain can correct steady-state errors that can cause the mBot to oscillate and overshoot if too strong.

In our code, we decided to implement this by having a global variable `[integral]`, which stores the summation of the errors over time. `[integral += error;]` To prevent integral windup, we decided to implement an upper limit using a global variable `[maxIntegral]` as shown below.

```
// Limit integral to prevent integral windup
if (integral > maxIntegral) integral = maxIntegral;
if (integral < -maxIntegral) integral = -maxIntegral;
```

Figure 7.2.4: Integral Windup

While K_I might be not be as relevant in our maze, as there are no obstructions, we still decided to implement this, to obtain a complete PID system.

7.2.5 Ziegler-Nichols Closed Loop Method

To ensure a structured method of obtaining the Proportional (K_P), Integral (K_I) and Derivative (K_D) Gains, we decided to utilise the Ziegler-Nichols Closed Loop method.
First, we set K_D and K_I to 0 and gradually raised the K_P value, until sustained oscillations occurred. This K_P value is also known as the ultimate gain, and it denoted by K_u.
Ultimate Gain, $K_u = 13.333$ $(3dp)$
Next, we had to find the period, T_u, of the oscillations and the results are as shown below.

Oscillations	Time (s)	Period (1/s)
5	4.89	0.9780
5	4.86	0.9720
6	5.83	0.9717
6	5.84	0.9733
7	6.85	0.9786
Period:		0.97471429

Table 7.2: Results for period of steady-state oscillations

Period, $T_u = 0.975$ $(3dp)$

Based on the Closed Loop Method, we can determine the K_P, K_D and K_I values.

$$K_P = 0.6K_u = 7.98$$
$$K_I = \frac{1.2K_u}{T_u} = 16.37$$
$$K_D = 0.075K_uT_u = 0.973$$

However, we realised that the Integral Gain was too high, and this can cause the correction to be too large and be inconsistent. As shown in Figure 7.2.5, Integral Component of PID where, the integral is being multiplied by K_I, is very large, and can cause the correction to be very unpredictable. Therefore, we decided to bring this value down to have a minimal effect on the system.

```
// Calculate correction using PID formula
float correction = (Kp * error) + (Ki * integral) + (Kd * derivative);
```

Figure 7.2.5: Correction Equation using PID

For a smooth and stable PID, we had to scale the gains, in the sequence shown in Figure 7.2.6. This ensured that our mBot was able to quickly return to the centre of path, as well as able to do so with minimum oscillation.

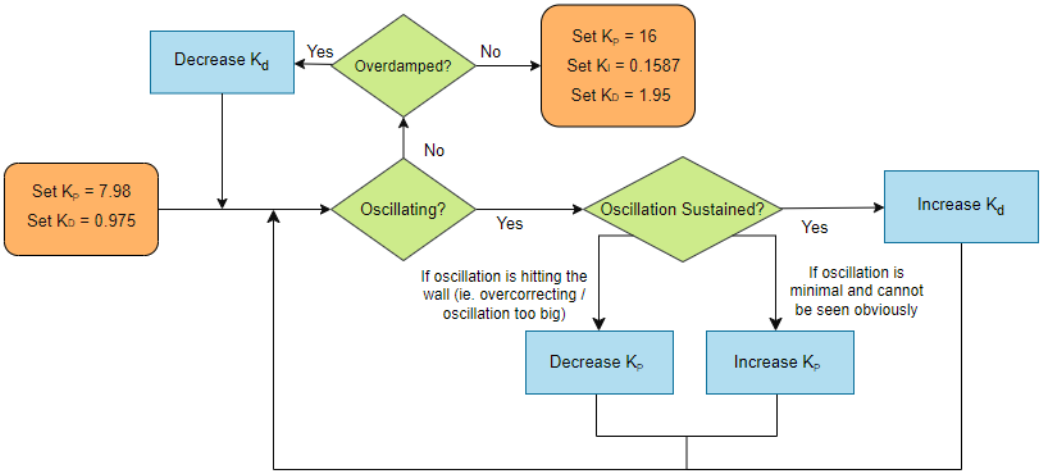


Figure 7.2.6: Sequence to obtain PID gains

After testing multiple gain values, our final K_P, K_D and K_I values were:

PID Gains	Pre-scaling	After Scaling
K_p	7.98	16
K_i	16.37	0.1587
K_d	0.973	1.95

Table 7.3: Final PID values

7.2.6 Final Code for PID

```
/************* PID Control Constants ***************/
float Kp = 16;           // Proportional gain for PID
float Ki = 0.1587;       // Integral gain for PID
float Kd = 1.95;         // Derivative gain for PID


/************* PID Control Initialisations ***************/
float targetHeading = 8;   // Desired heading angle or target distance
float error = 0;           // Difference between target and current heading
float previousError = 0;   // Error from the previous cycle
float integral = 0;        // Sum of all previous errors for integral term
float maxIntegral = 100.0; // Maximum limit for integral to prevent "integral windup"
float derivative = 0;      // Rate of change of error

/************* Functions to move robot using PID control ***************/
void moveRobot(float currentHeading) {
  error = targetHeading - currentHeading;       // Error as difference from target
  derivative = error - previousError;           // Derivative as error change rate
  integral += error;                            // Accumulate error for integral term

  // Limit integral to prevent integral windup
  if (integral > maxIntegral) integral = maxIntegral;
  if (integral < -maxIntegral) integral = -maxIntegral;

  // Calculate correction using PID formula
  float correction = (Kp * error) + (Ki * integral) + (Kd * derivative);

  // Adjust motor speeds based on correction
  motor1Speed = base1Speed - correction;
  motor2Speed = base2Speed + correction;

  // Apply speeds to motors
  motor1.run(-motor1Speed); // Inverse direction for motor1
  motor2.run(motor2Speed);  // Forward direction for motor2

  previousError = error;    // Store current error for next loop iteration
  delay(10);
}
```

7.3 IR Proximity Sensing and Calculations

7.3.1 Understanding IR Sensor Fundamentals

We are using indirect incidence setup to transmit IR from the emitter to the detector. Based on Figure 7.3.1, we are using analogue input pin A0 of the mBot to measure the analogue voltage across the detector.

As the mBot approaches the maze wall, the IR receiver would receive more IR which increases its resistance. Since it is connected to a 8.2kΩ resistor, we have a potential divider circuit. The voltage across the IR detector would thus increase, resulting in a lower sensor reading.

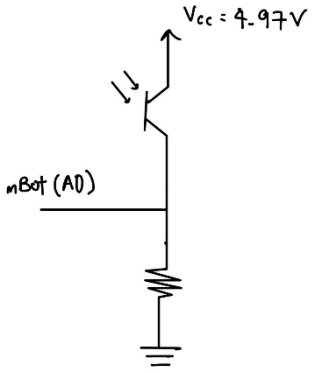


Figure 7.3.1: IR Circuit

When the mBot is further away from the wall, there is lesser IR, thus the resistance of the IR detector decreases resulting in a higher sensor reading.

7.3.2 Obtaining IR Distance

To map the sensor value of the IR Receiver to the actual distance, we must understand how the sensor output voltage correlates with the distance of an object reflecting the IR light. By placing a ruler between the wall and the IR sensor circuit, we can map each distance to the sensor value output by the analogue pin A0. We collected sensor values for distance ranging from 1cm to 10cm.

distance	sensor_out
1	883
2	860
3	841
4	795
5	685
6	633
7	589
8	558
9	544
10	525

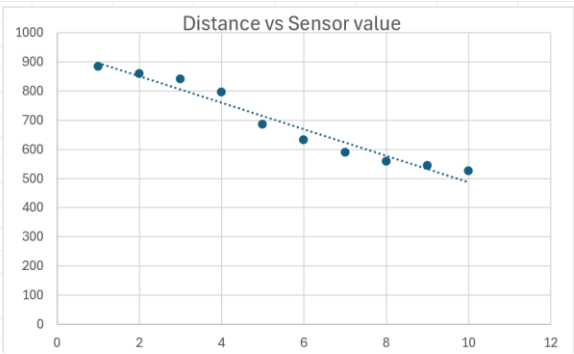


Figure 7.3.2: Data from IR Detector

Based on the data plotted, the sensor output and distance appear to exhibit an inverse relationship. Given that the measurements are taken over a narrow distance range of 1 to 15, we approximate the relationship by assuming that the distance is inversely proportional to the sensor output. Thus, the distance (d) and sensor output (V) can be model as:

$$d = m\left(\frac{1}{V}\right) + c$$

Using the formula, we can map the values to the distance by plotting the graph on excel and finding the equation relating distance to the Sensor value, taking d as the y-axis and 1/V as the x-axis.

distance	sensor_out	1/sensor
1	883	0.001133
2	860	0.001163
3	841	0.001189
4	795	0.001258
5	685	0.00146
6	633	0.00158
7	589	0.001698
8	558	0.001792
9	544	0.001838
10	525	0.001905

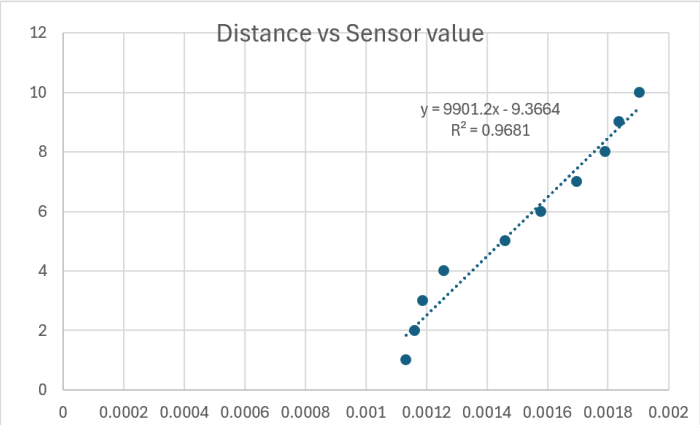


Figure 7.3.3: Linearising Data Points

After finding the equation to map the sensor value, we find that the $R^2 = 0.9681$ which indicates a strong linear correlation between x and y. Thus, we concluded that the model is a good fit to map the Sensor Values to distance and utilise the equation to transform the Sensor values into actual distance in the code. By mapping the distance to the sensor value, it makes It easier to understand and interpret results. Furthermore, it standardizes the readings across the whole code allowing for easier comparison between the ultrasonic sensor and the IR sensor since both sensors are measuring the distance between the mBot and the maze wall.

7.3.3 Initial Approach to Navigate

Our initial approach makes use of the mapped value of the IR sensor value to determine the proximity of a wall. Using the same analysis on the mapping equation in section 7.3.2, we find that the initial equation was:

$$d = 11136.9\left(\frac{1}{V}\right) - 10.874$$

```
// Function to read and calibrate distance from infrared sensor
float readInfrared() {
  digitalWrite(DECODER_A_PIN, HIGH);  // Set decoder pins to read infrared signal
  digitalWrite(DECODER_B_PIN, HIGH);
  delay(10);
  int sensorValue = analogRead(IR_PIN);  // Get raw analog value from IR sensor
  delay(10);                            // Delay for stability in readings
  float a = 11136.9;                     // Calibration constant (determined experimentally)
  float b = -10.874;                     // Calibration constant (determined experimentally)

  float calibrated_distance = (a / sensorValue) + b;  // Calibrate distance using formula
  return calibrated_distance;
}
```

Figure 7.3.4: readinfrared() function

We use the distance measured by the IR sensor when the mBot is positioned in the middle of two walls as a reference point, calling this distance the *irHeading*. By comparing the current sensor reading to the *irHeading*, we can determine whether the mBot is centered.

However, this approach does not account for environmental changes. Factors like ambient lighting and nearby electronic devices can influence the presence of IR in different areas of the maze. These variations can cause inconsistencies in the voltage across the IR detector, reducing the accuracy of the measurements and limiting the system's reliability.

7.3.4 Implementation of Ambient IR

To tackle the changes in ambient lightings, we must regularly measure the amount of IR present in the environment. This is achieved by turning the emitter off and reading the analogue sensor value of the detector. This value is known as the “baseline” voltage. We store the value of the “baseline” voltage in a variable named “ambientValue”.
The steps in determining proximity using IR while accounting for the ambient light is:

1. Read the sensor value when emitter is on (Sensor Value)
2. Read sensor value when emitter is off (Ambient Value)
3. The difference between the Sensor Value and the Ambient Value corresponds to a more accurate reading of the IR detector at a particular distance.

We must recalibrate the mapping equation of the sensor value that is independent of the ambient light and the distance. To achieve this, we added additional steps by taking the difference between the sensor value and the ambient value and store it in a variable called “noAmbient”. Using the same analysis on the mapping equation in *Section 7.3.2*, we find that the new equation is:

$$d = 9901.2\left(\frac{1}{V}\right) - 9.6664$$

```
33  float readInfrared() {
34    digitalWrite(DECODER_A_PIN, HIGH);  // Set decoder pins to read infrared signal
35    digitalWrite(DECODER_B_PIN, HIGH);
36    delay(10);
37    int sensorValue = analogRead(IR_PIN);  // Get raw analog value from IR sensor
38    delay(10);                            // Delay for stability in readings
39    float a = 9901.2;                     // Calibration constant (determined experimentally)
40    float b = -9.3664;                     // Calibration constant (determined experimentally)
41
42    int ambientValue = findAmbient();
43
44    int noAmbient = sensorValue - ambientValue;
45    float calibrated_distance = (a / noAmbient) + b;  // Calibrate distance using formula
46    return calibrated_distance;
47  }
```

Figure 7.3.5: Improved readinfrared() function

Using the same concept as *Section 7.3.3*, we make use of the irHeading and the sensor value that is independent of the ambient light to find the deviation. Depending on the deviation, the mBot will attempt to steer back to its original course.

7.3.5 Final Code for IR Proximity Sensing

```
int findAmbient(){
  digitalWrite(DECODER_A_PIN, HIGH);  // Set decoder pins to read infrared signal
  digitalWrite(DECODER_B_PIN, LOW);
  delay(10);
  int ambientValue = analogRead(IR_PIN);
  delay(10);
  return ambientValue;
}

// Function to read and calibrate distance from infrared sensor
float readInfrared() {
  digitalWrite(DECODER_A_PIN, HIGH);  // Set decoder pins to read infrared signal
  digitalWrite(DECODER_B_PIN, HIGH);
  delay(10);
  int sensorValue = analogRead(IR_PIN);  // Get raw analog value from IR sensor
  delay(10);                            // Delay for stability in readings
  float a = 9901.2;                     // Calibration constant (determined experimentally)
  float b = -9.3664;                     // Calibration constant (determined experimentally)

  int ambientValue = findAmbient();

  int noAmbient = sensorValue - ambientValue;
  float calibrated_distance = (a / noAmbient) + b;  // Calibrate distance using formula
  return calibrated_distance;
}

// Function to read and store both infrared and ultrasonic distances
void readSensors(float* infrared_dist, float* ultrasonic_dist) {
  *infrared_dist = readInfrared();   // Store infrared distance in passed variable
  *ultrasonic_dist = readUltrasonic(); // Store ultrasonic distance in passed variable
}
```

7.4 Colour Detections and Implementing K-Nearest Neighbours

7.4.1 Obtaining RGB values

The Light Dependent Resistor (LDR) is a type of resistor that operates based on the principle of photoconductivity, where its resistance varies with the intensity of light. As the light intensity increases, the resistance across it decreases. This corresponds to a decrease in the voltage across the LDR.

When RGB (Red, Green, Blue) LEDs illuminate a coloured surface, the surface reflects varying amounts of light for each colour. This change in the amount of light reflected for each primary colour light, can be detected by the LDR, which causes its resistance value to change. This change in the voltage is then detected by the Analog pin and can then be scaled to the corresponding RGB values.

We did this by first obtaining the raw values detected by the Analog pin A1, and then converting to the respective RGB values of the colour. (Refer to Figure 7.4.1)

Furthermore, we utilised the Black and Grey Difference Arrays introduced in Studio 12 to normalise the colour readings from the sensor. This ensured accurate and consistent measurements, irrespective of environmental factors or sensor variations. The formula for the normalisation of the colour is as follows:

$$Normalized\ Value\ = \left(\frac{Raw\ Reading\ \ -\ Black\ Calibration\ Value}{Grey\ Difference\ Value}\right) \times 255$$

```
// Function to normalize the color readings based on black and grey calibration arrays
void readNormalizedColor(int color[]) {
  color[0] = (find_RED() - blackArray[0]) / (greyDiff[0]) * 255;   // Normalize red value
  color[1] = (find_GREEN() - blackArray[1]) / (greyDiff[1]) * 255; // Normalize green value
  color[2] = (find_BLUE() - blackArray[2]) / (greyDiff[2]) * 255;  // Normalize blue value
  return color;  // Return the normalized color array
}
```

Figure 7.4.1: Scaling Raw values to RGB

To increase accuracy, we also obtained the average of 5 different sets of reading from the LDR, using the getAverage() function, before determining the colour.

```
// Helper function to get an average reading from LDR sensor
int getAvgReading(int times) {
  int total = 0;
  for (int i = 0; i < times; i++) {
    total += analogRead(LDR);        // Read LDR value and add to total
    delay(LDRWait);                  // Delay between readings for stabilization
  }
  return total / times;              // Return the average reading
}
```

Figure 7.4.2: Obtaining average LDR values

Additional delays [LDRWait and RGBWait] were also added to ensure that the LDR voltage stabilises before reading it.

7.4.2 Initial Approach

Upon Reading the RGB values, we would analyse each primary colour value, to see if they fall within the range for the colour. This is done using simple if-else conditional statements to compare the obtained RGB value with the ranges for each if-else block.

However, with the changes in the ambient lighting, we had to recalibrate and modify these values every time. Therefore, we realised that this method might not be as accurate, and doing so would be significantly tedious. The code was very prone to bugs and logic error if we calibrated the colour sensor incorrectly. Therefore, we decided to utilise the K-Nearest Neighbours (KNN) Method, to detect the colour under the mBot. This method was much more efficient and far more elegant.

```
//CONDITIONAL STATEMENTS TO FIND COLOUR
if ((red > 225) && (green < 50) && (blue < 40)){
  return RED;
}
if ((red < 60) && (green > 220) && (blue < 34)){
  return GREEN;
}
if ((red > 170) && (green < 100) && (blue < 40)){
  return ORANGE;
}
if ((red < 190) && (green < 196) && (blue < 184)){
  return PINK;
}
if ((red < 74) && (green < 190) && (blue > 220)){
  return LIGHTBLUE;
}
return WHITE;
```

Figure 7.4.3: Obtaining average LDR values

7.4.3 K-Nearest Neighbours (KNN) Implementation

The K-Nearest Neighbours Algorithm is a non-parametric, supervised learning classifier, which uses proximity to classify or predict the category of an individual data point. This algorithm can be used to compare a test point to known training data, and it identifies the closest match based on similarity. Our general idea to implement this was to read the RGB of the colour paper under the mBot and compare them with a predefined training data. This allowed us to determine the closest matching waypoint colour (Red, Green, Blue, Orange, Pink and White).

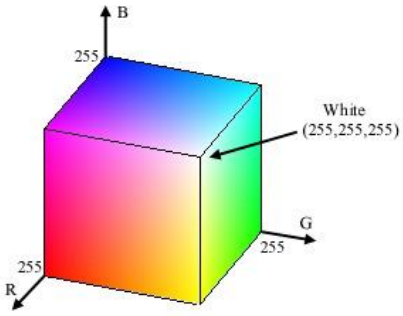


Figure 7.4.4: 3D RGB Colour Space

We first measured and recorded the RGB values for each waypoint, storing them in arrays corresponding to each colour. These predefined values will then be used later to compare against the detected colour and determine the closest match.

```
/************* Color Detection Reference Arrays ****************/
float whiteArray[] = {198, 255, 257};   // Reference values for white
float blackArray[] = {629, 821, 778};   // Reference values for black
float greyDiff[] = {120, 95, 75};       // Differences for normalization
float redArray[] = {170, 60, 34};       // Reference values for red
float greenArray[] = {60, 189, 97};     // Reference values for green
float orangeArray[] = {165, 120, 37};   // Reference values for orange
float pinkArray[] = {191, 196, 184};    // Reference values for pink
float blueArray[] = {72, 190, 220};     // Reference values for blue
```

Figure 7.4.5: Color Detection Reference Array

To find the distance between any two points in a 3D space, we can calculate the Euclidean distance as follows:

$$Distance\ between\ two\ points = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2}$$

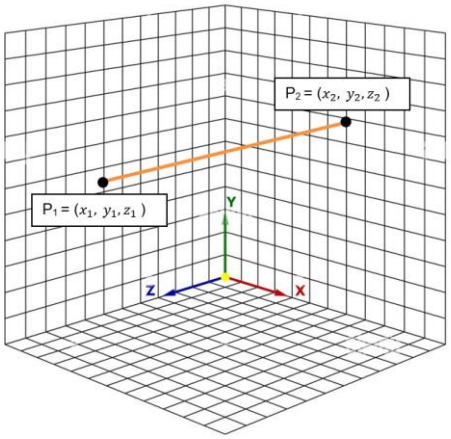


Figure 7.4.6: 2 Points in a 3D space

Using this method, we calculated the distance between the detected color and each waypoint colour within the 3D RGB space. In our code, we called a function distance(), which takes the

RGB values of both the detected colour and the waypoint colour as parameters *(refer to Figure 7.4.5: Color Detection Reference Array).*

```
// Function to calculate Euclidean distance between two colors in 3D color space
float distance(int x1, int y1, int z1, int x2, int y2, int z2) {
  return sqrt(square(x2 - x1) + square(y2 - y1) + square(z2 - z1));
}
```

Figure 7.4.7: Function for finding Euclidean distance

Subsequently, the distances between the detected colour and each waypoint were stored in an array called distances[]. Using a loop, the code performs a linear search to find the shortest distance within the distances[] array. Finally, it returns the waypoint colour that is closest to the detected colour. This result is then passed to the main loop function to execute the corresponding task associated with the identified colour.

```
// Calculate distances between current color and each predefined color
float distances[] = {
  distance(redArray[0], redArray[1], redArray[2], currentColor[0], currentColor[1], currentColor[2]),
  distance(greenArray[0], greenArray[1], greenArray[2], currentColor[0], currentColor[1], currentColor[2]),
  distance(orangeArray[0], orangeArray[1], orangeArray[2], currentColor[0], currentColor[1], currentColor[2]),
  distance(pinkArray[0], pinkArray[1], pinkArray[2], currentColor[0], currentColor[1], currentColor[2]),
  distance(blueArray[0], blueArray[1], blueArray[2], currentColor[0], currentColor[1], currentColor[2]),
  distance(whiteArray[0], whiteArray[1], whiteArray[2], currentColor[0], currentColor[1], currentColor[2])
};

int colors[] = {RED, GREEN, ORANGE, PINK, BLUE, WHITE};  // Array representing color codes

// Find the color with the minimum distance
int minColor = colors[0];      // Initialize to minimum distance and its corresponding minimum colour
float minDist = distances[0];
for (int i = 1; i < 6; i++) {
  if (distances[i] < minDist) {  // Check if current distance is smaller and update accordingly
    minDist = distances[i];
    minColor = colors[i];
  }
}
return minColor;
}
```

Figure 7.4.8: Linear search to find shortest distance

7.4.4 Calibration

During the Final Run, we were required to navigate the mBot through 2 separate mazes, at different locations of the lab. Therefore, we decided to perform the calibration to find the waypoint RGB colours at 3 different locations in the lab, and subsequently find the average. This approach ensured that our mBot could operate effectively under varying light conditions and across different locations.

Waypoint Colour	RED				Green				Blue			
	Test 1	Test 2	Test 3	Avg (R)	Test 1	Test 2	Test 3	Avg (G)	Test 1	Test 2	Test 3	Avg (B)
White	200	198	196	198	260	250	255	255	255	260	256	257
Red	172	168	170	170	62	59	60	60.333	33	36	34	34.333
Green	58	62	60	60	190	188	189	189	95	98	97	96.667
Orange	167	163	165	165	121	119	120	120	40	35	37	37.333
Pink	190	192	191	191	195	197	196	196	183	185	184	184
Blue	70	74	72	72	191	189	190	190	222	218	220	220

Table 7.4 : Finding average RGB values of waypoint

Additionally, we repeated the calibration on the day of the Final Run as well, to account for any changes in lighting conditions that might have occurred since the previous calibration. This precaution helped to maintain the reliability and consistency of the mBot's performance.

7.4.5 Final Code

Initialisation

```
/************* Pin Numbers ***************/
#define DECODER_A_PIN 11 // Decoder pin A for color sensor
#define DECODER_B_PIN 12 // Decoder pin B for color sensor
#define LDR A1           // Pin for the Light Dependent Resistor (LDR) sensor

/************* Colour Definitions ***************/
#define RED 1
#define GREEN 2
#define ORANGE 3
#define PINK 4
#define BLUE 5
#define WHITE 6

/************* Stabilisation Delays ***************/
#define RGBWait 50      // Delay for RGB color stabilization in milliseconds
#define LDRWait 100      // Delay for LDR reading stabilization in milliseconds


/************* Color Detection Reference Arrays ***************/
float whiteArray[] = {198, 255, 257};   // Reference values for white
float blackArray[] = {629, 821, 778};   // Reference values for black
float greyDiff[] = {120, 95, 75};       // Differences for normalization
float redArray[] = {170, 60, 34};       // Reference values for red
float greenArray[] = {60, 189, 97};     // Reference values for green
float orangeArray[] = {165, 120, 37};   // Reference values for orange
float pinkArray[] = {191, 196, 184};    // Reference values for pink
float blueArray[] = {72, 190, 220};     // Reference values for blue
```

Finding RGB values

```
// Helper function to get an average reading from LDR sensor
int getAvgReading(int times) {
  int total = 0;
  for (int i = 0; i < times; i++) {
    total += analogRead(LDR);        // Read LDR value and add to total
    delay(LDRWait);                  // Delay between readings for stabilization
  }
  return total / times;              // Return the average reading
}

// Function to get the average red color reading
int find_RED() {
  digitalWrite(DECODER_A_PIN, LOW);  // Set decoder to read red component
  digitalWrite(DECODER_B_PIN, LOW);
  delay(RGBWait);                    // Wait for stabilization
  return getAvgReading(5);           // Return average reading over 5 samples
}

// Function to get the average green color reading
int find_GREEN() {
  digitalWrite(DECODER_A_PIN, HIGH);  // Set decoder to read green component
  digitalWrite(DECODER_B_PIN, LOW);
  delay(RGBWait);                     // Wait for stabilization
  return getAvgReading(5);            // Return average reading over 5 samples
}

// Function to get the average blue color reading
int find_BLUE() {
  digitalWrite(DECODER_A_PIN, LOW);   // Set decoder to read blue component
  digitalWrite(DECODER_B_PIN, HIGH);
  delay(RGBWait);                     // Wait for stabilization
  return getAvgReading(5);            // Return average reading over 5 samples
}

// Function to normalize the color readings based on black and grey calibration arrays
void readNormalizedColor(int color[]) {
  color[0] = (find_RED() - blackArray[0]) / (greyDiff[0]) * 255;   // Normalize red value
  color[1] = (find_GREEN() - blackArray[1]) / (greyDiff[1]) * 255; // Normalize green value
  color[2] = (find_BLUE() - blackArray[2]) / (greyDiff[2]) * 255;  // Normalize blue value
  return color;  // Return the normalized color array
}
```

Find closest waypoint and return

```
// Function to calculate Euclidean distance between two colors in 3D color space
float distance(int x1, int y1, int z1, int x2, int y2, int z2) {
  return sqrt(square(x2 - x1) + square(y2 - y1) + square(z2 - z1));
}


// Function to detect the closest matching color by comparing current color with known colors
int detectColor() {
  int currentColor[3];                    // Array to hold current normalized color values
  readNormalizedColor(currentColor);         // Get normalized color values

  // Print current color values for debugging
  Serial.println(currentColor[0]);
  Serial.println(currentColor[1]);
  Serial.println(currentColor[2]);

  // Calculate distances between current color and each predefined color
  float distances[] = {
    distance(redArray[0], redArray[1], redArray[2], currentColor[0], currentColor[1], currentColor[2]),
    distance(greenArray[0], greenArray[1], greenArray[2], currentColor[0], currentColor[1], currentColor[2]),
    distance(orangeArray[0], orangeArray[1], orangeArray[2], currentColor[0], currentColor[1], currentColor[2]),
    distance(pinkArray[0], pinkArray[1], pinkArray[2], currentColor[0], currentColor[1], currentColor[2]),
    distance(blueArray[0], blueArray[1], blueArray[2], currentColor[0], currentColor[1], currentColor[2]),
    distance(whiteArray[0], whiteArray[1], whiteArray[2], currentColor[0], currentColor[1], currentColor[2])
  };

  int colors[] = {RED, GREEN, ORANGE, PINK, BLUE, WHITE};  // Array representing color codes

  // Find the color with the minimum distance
  int minColor = colors[0];      // Initialize to minimum distance and its corresponding minimum colour
  float minDist = distances[0];
  for (int i = 1; i < 6; i++) {
    if (distances[i] < minDist) {  // Check if current distance is smaller and update accordingly
      minDist = distances[i];
      minColor = colors[i];
    }
  }
  return minColor;
}
```

7.5 Movement of mBot

This section will tie up the algorithms explained earlier and elaborate on how we utilised this to control the movement of the mBot.

The movement of the mBot depends on the detection of the black strip of paper placed at waypoint challenges.

```
// Black Line Detection Function
bool on_black_line() {
  int sensorState = lineFinder.readSensors();  // Read state of line-following sensors
  // Return true if the sensor detects a black line (specific sensor states)
  return (sensorState == S1_IN_S2_IN || sensorState == S1_OUT_S2_IN || sensorState == S1_IN_S2_OUT);
}
```

Figure 7.5.1: Boolean Function to check if a black strip of paper is detected

The detection of the black strip is done by the Line Follower Sensor. As illustrated in Figure 7.5.1, there are three cases which we consider as that the mBot has reached a waypoint challenge. For purposes of reference, S1 refers to Sensor 1, and S2 refers to Sensor 2.

A. Both S1 and S2 detect the black strip of paper.
B. S2 detects the black strip of paper, but S1 does not detect it.
C. S1 detects the black strip of paper, but S2 does not detect it.

While it may be sufficient to consider just Case A, we had to consider the worst-case scenarios - wherein the mBot enters the waypoint challenges at a skewed angle. In such situations, either Case B or C may be invoked.

The mBot then acts accordingly to two distinct situations, first when a black strip is detected, and when a black strip is not detected.

7.5.1 Black Strip has been detected

```
if (on_black_line()) {
 stopMotor();
 int detectedColor = detectColor();
  switch (detectedColor) {
    case RED: turnLeft(); break;
    case GREEN: turnRight(); break;
    case ORANGE: uTurn(); break;
    case PINK: doubleLeftTurn(); break;
    case BLUE: doubleRightTurn(); break;
    case WHITE: celebrate(); break;
  }
}
```

Figure 7.5.2: Algorithm used when mBot reaches waypoint challenges

In this case, the mBot has reached a waypoint challenge. The mBot stops as invoked by the stopMotor() function. The Colour Sensing Circuit is then activated via a series of functions. Through the KNN algorithm, the mBot will be able to identify the colour and execute the respective action.

Colour	Action
Red	Left Turn
Green	Right Turn
Orange	U-Turn
Pink	2 Successive Left Turns
Light Blue	2 Successive Right Turns
White	End of Run / Celebrate

Table 7.5: Waypoint Challenges

The turns were executed through a concept known as Pulse-Width Modulation (PWM) Differential. The speed of a motor depends on the duty cycle of a PWM Signal – the higher the duty cycle, the greater the power supplied to the motor, and hence, the higher the speed of the motor. By varying the duty cycle of the PWM Signal sent to the two motors, an imbalance in speed is created between the two motors. This allows for the mBot to turn in the desired direction.

Left Turn: Red

```
// Function to turn the robot left
void turnLeft() {
  motor1.run(base1Speed);              // Motor 1 runs forward at base speed
  motor2.run(base2Speed + speed_gain); // Motor 2 speed increased for sharper turn
  delay(right_left_dur);               // Delay to control turn duration
}
```

Figure 7.5.3: turnLeft() Function

Purpose	Variable names	Values
Base Speed of Left Motor	base1Speed	170
Base Speed of Right Motor	base2Speed	150
PWM Differential	speed_gain	80
Duration of Turn	right_left_dur	362 (in milliseconds)

Table 7.6: Defined Values in the turnLeft() function

To execute a left turn, the right motor must be faster than the left motor. Hence, the right motor is given a higher duty cycle (as provided by *speed_gain*) as compared to the left motor. This allows the right motor to spin faster than the left motor, executing a left turn. By trial and error, we figured out that the duration for an exact 90-degree turn was 362 milliseconds.

It should also be noted that the base speeds of the left and right motors were kept at 170 and 150, to ensure that the overall base speed does not exceed 255 after the addition of the *speed_gain*.

Right Turn: Green

This follows the same algorithm as the turnLeft() function, except that the left motor is made to be faster than the right motor, to execute a right turn.

U-Turn: Orange

```
// Function to perform a U-turn
void uTurn() {
  motor1.run(base1Speed + speed_gain);  // Motor 1 runs forward with increased speed
  motor2.run(base2Speed);               // Motor 2 runs forward at base speed
  delay(uturn_dur);                     // Delay to complete a full U-turn
}
```

Figure 7.5.4: uTurn() Function

Purpose	Variable names	Values
Duration for U-Turn	uturn_dur	700 (in milliseconds)

Table 7.7: Defined Values in the uTurn() function

This algorithm is similar to the turnLeft() function as well. The left motor runs faster than the right motor, to spin the mBot in the clockwise direction. A longer duration of 700 milliseconds is given to allow the mBot to rotate 180 degrees perfectly.

Two Successive Left Turns: Pink

```
// Function to perform a double left turn sequence
void doubleLeftTurn() {
  turnLeft();       // Perform first left turn
  delay(60);
  moveForward();    // Move forward briefly to reposition
  delay(1150);      // Wait for movement stabilization
  stopMotor();      // Stop motors to prevent overshoot
  turnLeft();       // Perform second left turn
  delay(50);

}
```

Figure 7.5.5: doubleLeftTurn() Function

The above algorithm builds up on Figure 7.5.3. The mBot turns left (called by the turnLeft() function) for a duration of 60 milliseconds. Then, it moves forward for 1150 milliseconds (called by the moveForward() function), and stops (called by the stopMotor() function). It turns left again for another 50 millesconds, before continuing its course. These values were obtained via trial-and-error.

Two Successive Right Turns: Light Blue

The algorithm for this movement is similar to Figure 7.5.5 except that the mBot executes two right turns instead.

7.5.2 Black Strip has not been detected

When no black strips are detected, by default, the mBot continues moving forward. The Ultrasonic Sensor and IR Detector ensure that the mBot moves in a straight line. Using the PID Algorithm *(explained in Section 7.2)*, the Ultrasonic sensor detects distances between the mBot and the right wall. In the event there is no right wall, the algorithm switches to the use of the IR Detector.

```
// Function to slightly adjust the robot to the left
void nudgeLeft() {
  motor1.run(-base1Speed + 20);         // Slightly reduce Motor 1 reverse speed
  motor2.run(base2Speed + 30);          // Slightly increase Motor 2 speed
  delay(100);
}

// Function to slightly adjust the robot to the right
void nudgeRight() {
  motor1.run(-base1Speed - 20);         // Slightly increase Motor 1 reverse speed
  motor2.run(base2Speed - 10);          // Slightly reduce Motor 2 speed
  delay(100);
}
```

Figure 7.5.6: nudgeLeft() and nudgeRight() Functions

Readings from the IR Detector invokes two functions, as shown in Figure 7.5.7. When the mBot moves too close to the left wall, the nudgeRight() function is invoked. When the mBot moves significantly further from the left wall, the nudgeLeft() function is called. These functions utilize PWM Differential to re-orientate the mBot back to a straight course. Furthermore, we calculate the deviation to determine whether the robot is within the target heading or out of it.

```
void loop() {
  float infrared_dist, ultrasonic_dist;
  readSensors(&infrared_dist, &ultrasonic_dist);
  float deviation = infrared_dist - irHeading;

  if (ultrasonic_dist > usThreshold) {
      if (deviation < 0.0) nudgeRight();
      else if (deviation > 0.0) nudgeLeft();
      else moveForward();
  } else {
      moveRobot(ultrasonic_dist);
  }
```

Figure 7.5.6: Using nudgeLeft() and nudgeRight() to navigate

7.6 Celebrate

```
101
102  // Function to trigger a celebratory action or tune
103  void celebrate() {
104    // Code for playing celebratory tune or action
105      // keys and durations found in NOTES.h
106    for (int i = 0; i < sizeof(music_key) / sizeof(int); ++i) {
107      // quarter note = 1000 / 4, eighth note = 1000/8, etc. (Assuming 1 beat per sec)
108      const int duration = 40000 / music_duration[i];
109      buzzer.tone(MUSIC_PIN, music_key[i], duration);
110      delay(duration * 1.80);
111      buzzer.noTone(MUSIC_PIN);
112    }
113  }
114
```

Figure 7.6.1: Celebrate Function

The celebratory tune is triggered by the mBot when it determines that it has reached the finish line. The code utilizes the MeBuzzer library to generate tones for the melody, based on predefined note frequencies and durations stored in the Celebrate.h header file.

The melody for the celebratory tune, The Imperial March, is stored in two arrays: music_key and music_duration. The music_key array specifies the sequence of notes, where each note corresponds to a specific frequency, while the music_duration array defines the timing for each note in milliseconds, allowing for accurate representation of note lengths such as quarter notes or eighth notes.

When the celebrate() function is called, it iterates through the music_key and music_duration arrays using a for loop, playing each note sequentially. The tone() function generates the appropriate frequency on the buzzer, and a slight delay between notes ensures smooth transitions and proper timing.

Once the melody is complete, the noTone() function is used to stop the buzzer, ensuring a clean end to the music. This structured use of arrays simplifies the implementation and makes the code reusable for other melodies.

The choice of The Imperial March, a widely recognized tune, adds an engaging and celebratory element to the mBot's performance, making its success at the finish line more memorable.

8 Difficulties and Resolutions

8.1 The Great Overflow

```
7
8   const int irPin = A1; // Analog pin connected to the output of the IR
9   int count = 0;
10  uint8_t motor1Speed = 170;
11  uint8_t motor2Speed = 150;
12  uint8_t right_left_dur = 5000;
13  uint8_t uturn_dur = 700;
14  int speed_gain = 80;
15
```

Figure 8.1: Initial code showing incorrect use of uint8_t for timing variables

8.1.1 Problem

The integer overflow occurred because the variables right_left_dur and uturn_dur were originally declared as uint8_t, which is an 8-bit unsigned integer with a maximum value of 255. The assigned values, right_left_dur = 5000 and uturn_dur = 700, exceeded this limit, causing the variables to wrap around and store incorrect values. Specifically, right_left_dur wrapped to 136 (5000 % 256) and uturn_dur wrapped to 188 (700 % 256). This resulted in the mBot executing movements for much shorter durations than intended, leading to erratic and incorrect behavior during tasks like turning and U-turning.

8.1.2 Solution

The issue was resolved by changing the data types of right_left_dur and uturn_dur to long, which is a 32-bit integer. This data type has a significantly larger range, from -2,147,483,648 to 2,147,483,647, allowing it to easily accommodate the required values of 5000 and 700 without risk of overflow. After this change, the mBot correctly performed movements for the intended durations, restoring its expected functionality.

8.1.3 Reflections

This issue highlighted the importance of selecting appropriate data types in embedded systems programming. Smaller data types like uint8_t are memory-efficient but should only be used when their range can safely handle the values being assigned. Changing to long not only resolved the immediate issue but also provided additional flexibility for future modifications that may require handling larger values. This experience reinforced the need for careful planning and debugging to avoid overflow issues and ensure the reliability of the mBot's operations.

8.2 The Faulty Foot

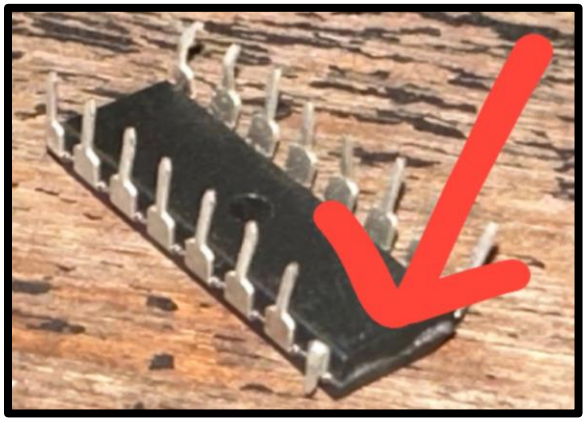


Figure 8.2: The decoder IC with the faulty Y3 Pin

8.2.1 Problem

The Y3 output pin of the decoder IC, responsible for toggling the IR emitter, was not properly plugged into the breadboard. Without a proper connection, the Y3 pin failed to send a LOW signal to turn the IR emitter off, leaving the emitter in a default ON state. As a result, the IR emitter remained constantly ON, making it impossible to calculate ambient light levels. This prevented the detector from distinguishing between emitted IR light and surrounding light, leading to inaccurate readings and navigation failures.

8.2.2 Solution

The Y3 output pin was securely reconnected to the breadboard, ensuring it could deliver the necessary signal to toggle the IR emitter on and off. With the connection restored, the IR emitter functioned as intended, enabling the mBot to calculate ambient light levels properly. This restored the detector's ability to detect distances and navigate effectively.

8.2.3 Reflections

A disconnected output pin, like the Y3 pin of the decoder IC, can cause significant disruptions in circuit functionality, even if it appears to be a minor issue. Ensuring all connections are secure is critical to maintaining proper signal flow and avoiding unpredictable behavior in the circuit. Systematic testing and debugging help identify subtle hardware issues, such as a disconnected pin, and restore full functionality.

This highlights the importance of thorough inspections and reliable connections in hardware design and assembly.

8.3 The Battery Betrayal



Figure 8.3: The Betrayal

8.3.1 Problem

The mBot was fully calibrated and had successfully completed multiple test runs. However, after working on it continuously for 3 hours, we grew concerned about the battery running out during the final assessment. To avoid this, we decided to charge it for 15 minutes while we went for lunch. After charging, the mBot's colour readings became erratic, and some previously calibrated colours were misdetected. Despite reconfiguring and salvaging 6 colours, 2 were detected incorrectly during the final assessment, resulting in lost marks. This happened because the freshly charged battery altered the power conditions, leading to changes in the brightness of the LEDs and the RGB sensor's performance compared to when it was initially calibrated.

8.3.2 Solution

To ensure consistent power conditions, always fully charge the mBot before calibration and testing. A stable power supply to the LEDs and RGB sensor ensures reliable operation. After charging the battery, allow it to stabilize for a few minutes, as freshly charged batteries may experience slight voltage fluctuations. If recharging the battery is unavoidable between tests, make sure to recalibrate the RGB sensor. This accounts for any changes in LED brightness or sensor readings caused by the freshly charged battery

8.3.3 Reflections

Calibrating a mBot on a low battery can lead to mismatched performance once the battery is recharged, as power conditions such as voltage and current levels change. For example, a freshly charged battery can make LEDs brighter, altering the reflections detected by the RGB sensor and causing erratic readings.

To ensure consistent sensor readings, calibration, testing, and operation should always be performed under the same power conditions. It is also essential to allocate extra time for recalibration if the mBot needs to be recharged between runs. Maintaining consistency in power conditions is crucial for accurate sensor readings and stable operation.

Sometimes, life presents unexpected challenges, like a perfectly calibrated mBot suddenly malfunctioning. These moments teach us the importance of adaptability, resilience, and learning to perform under pressure, even when things don't go as planned.

9 | Work division of the project

The initial building of the mBot was carried out by Justin and Kelvin, who assembled the mBot and ensured its hardware was ready for operation. Karthik and Vetri took charge of coding, setting up the foundational logic and ensuring smooth integration of all components.

Justin played a pivotal role in learning and integrating the PID control algorithm into the mBot, which greatly enhanced its ability to navigate accurately and maintain consistent performance during the course. This implementation was instrumental in overcoming challenges such as erratic movements and improving the overall efficiency of the mBot.

Vetri was highly instrumental in the development and refinement of the code. He contributed significantly to the logic for the mBot's movement and sensor interactions, ensuring that the mBot was able to detect and respond to its environment effectively.

Karthik brought a dynamic approach to the team, helping in all areas as needed. He played a key role in ensuring the mBot's precision for turns, fine-tuning the movement logic to handle complex manoeuvrers with accuracy.

Kelvin demonstrated resourcefulness by proactively seeking guidance and input from teachers to resolve critical issues, such as identifying the short circuit caused by the metal chassis which prevented unnecessary delays and helped the team stay on track.

Throughout the project, all members actively contributed to refining the mBot's performance. This included collective efforts in brainstorming, testing, and troubleshooting to continuously improve the mBot's capabilities and ensure it met all the challenge requirements. The collaborative teamwork allowed the group to build a mBot that was not only functional but also innovative in its solutions.

10 | Conclusion

CG1111A has allowed us to not only sharpen our technical skills but also to develop a deeper understanding of teamwork, problem-solving, and resilience.

Throughout this project, we faced unexpected challenges that pushed us to think critically, adapt quickly, and rely on one another to find creative solutions.

We saw ourselves grow over the past few months—not just as aspiring engineers but also as individuals learning the value of collaboration and perseverance. Each setback became an opportunity to improve, and every success reminded us of the power of teamwork and dedication.

It has been a privilege to work alongside a group of talented friends and engineers, each bringing unique strengths to the table. This experience has shown us the importance of mutual support and the bond that forms when striving together toward a common goal.

As we reflect on our journey, we leave CG1111A with not just a completed project but a shared memory of growth, determination, and camaraderie that we will carry with us into our future endeavours.